earlier version, repeated here for comparison, the beneficiary sets the state of the auction:

```
function changeState(Phase x) public onlyBeneficiary {
        if (x < state) revert();
        state = x;
    }
```

The `changeState()` function as written here looks fine and will work well as long as the beneficiary explicitly and linearly advances the state value. It's appropriate for a traditional non-blockchain system in which you may redeploy the code from one auction to the next and as many times as you want. But a smart contract deployed with `changeState()` would be good for only a single use in a blockchain environment because the immutability requirement of the blockchain means that a smart contract, once deployed, cannot be overwritten! This limitation is significant.

Now consider the `advancePhase()` function from listing 6.1, which addresses this limitation of the earlier version by circularly advancing the state: `Init`, `Bidding`, `Reveal`, `Done`, and back to `Init`, ready for the next auction. It also emits the events `BiddingStarted` and `RevealStarted`.

> **NOTE** You must be aware of the immutable nature of smart contracts (and on-chain data) when you are designing blockchain-based systems. It's important to think of smart contracts as long-running programs and to make provisions for repeat runs through proper use of states.

EVENT LOG ON-CHAIN DATA

Now you can upload the smart contract to the Remix IDE and check the events in operation. Compile, run, and deploy BlindAuction.sol; then take a look at the Tx recorded in the console window. The part of the recording called *logs* generated